

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

BACHARELADO EM TECNOLOGIA DA INFORMAÇÃO

BANCO DE DADOS - IMD/UFRN

BIANCA BEZERRA PIRES - 20240020515

DANIEL SILVA MARTINS - 20240004914

THYAGO PEREIRA DA SILVA - 20230034295

PROJETO SEBO LITERÁRIO:

modelagem e implementação – iteração 2

NATAL/RN

2026

SUMÁRIO

1 INTRODUÇÃO E MINIMUNDO 3
1.1 Descrição do minimundo 3
2 EVOLUÇÃO DO MODELO: DA ITERAÇÃO 1 À ITERAÇÃO 2 3
3 DIAGRAMA ENTIDADE-RELACIONAMENTO E MODELO RELACIONAL 4
3.1 Dicionário de dados 5
3.2 Cardinalidades dos relacionamentos 6
4 ESTRUTURA DO BANCO DE DADOS (DDL) E POVOAMENTO (DML) 8
4.1 Esquema de definição de dados (DDL) 8
4.2 Povoamento e massa de testes (DML) 10
5 REGRAS DE NEGÓCIO IMPLEMENTADAS PELA APLICAÇÃO 12
5.1 Compra de um livro 12
5.2 Tentativa de comprar um livro que já foi vendido 12
5.3 Cancelamento de pedido 13
5.4 Aceite de um livro oferecido em troca 13
6 EXEMPLOS DE UTILIZAÇÃO E CONSULTAS PRÁTICAS 13
6.1 Consulta 1: catálogo com exemplares disponíveis e faixas de preço 14
6.2 Consulta 2: histórico de pedidos de um cliente 14
6.3 Consulta 3: faturamento e vendas por gênero 15
6.4 Consulta 4: doações e trocas e crédito gerado 15
6.5 Consulta 5: estoque disponível por estado de conservação 16

1 INTRODUÇÃO E MINIMUNDO

O Sebo Literário é um sistema web para gerenciar um sebo online de livros usados, com catálogo, carrinho e pedidos, doações e trocas e autenticação de usuários com perfis de administrador e de cliente. Este documento corresponde à Iteração 2 do projeto: parte da modelagem da Iteração 1 e a evolui para um banco de dados relacional executável em PostgreSQL 15+, com scripts de criação (DDL) e povoamento (DML), exemplos de transações e consultas analíticas. O repositório, a integração opcional com a aplicação e o roteiro de apresentação estão no documento complementar.

1.1 Descrição do minimundo

O acervo é organizado em um catálogo de obras, cada uma com seus autores, gênero e editora. Cada obra pode existir em vários exemplares físicos, cada um com estado de conservação, preço de venda e disponibilidade próprios, já que num sebo cada cópia física é individual: tem seu próprio estado e preço e só pode ser vendida uma vez.

Os clientes se cadastram com seus dados pessoais, endereço e telefones, e acessam o sistema com e-mail e senha. Cada usuário tem um perfil, de cliente ou de administrador. Os clientes podem comprar exemplares por meio de pedidos, que têm um status (pendente, pago, enviado, entregue ou cancelado) e são entregues no endereço informado. No momento da compra, o preço cobrado de cada exemplar é gravado no item do pedido, de modo que remarcações posteriores não alteram o histórico.

Além de comprar, o cliente pode doar livros ou oferecê-los em troca: cada solicitação reúne um ou mais livros, que são avaliados individualmente pelo sebo. Os aceitos passam a integrar o acervo como novos exemplares e, no caso da troca, geram crédito para compras futuras. Por fim, qualquer pessoa, cliente cadastrado ou visitante, pode enviar mensagens de contato ao sebo.

2 EVOLUÇÃO DO MODELO: DA ITERAÇÃO 1 À ITERAÇÃO 2

A Iteração 2 mantém a estrutura da Iteração 1: as mesmas 14 tabelas, os mesmos nomes e o mesmo padrão (tabelas em PascalCase, chave primária id_<tabela> e chave estrangeira com o nome da chave referenciada). As mudanças se limitam ao que era necessário para o sistema funcionar de ponta a ponta com o banco de dados. O Quadro 1 resume o que mudou e por quê.

Quadro 1 – O que mudou da Iteração 1 para a Iteração 2 e por quê

Tabela	Iteração 1	Iteração 2	Por que mudou
Exemplar	Sem atributo de preço.	Incluído preco_venda DECIMAL(10,2).	O catálogo precisa exibir um preço e o pedido precisa cobrá-lo. Na Iteração 1 o Item_Pedido guardava um preço, mas não havia de onde ele vir. O preco_venda é o preço de tabela e pode ser remarcado.
Item_Pedido	preco (float), titulo e quantidade.	preco_unitario DECIMAL(10,2); sem titulo e quantidade.	O preco_unitario guarda o valor cobrado no ato da compra e não muda se o exemplar for remarcado. O titulo era redundante (vem de Exemplar e Obra). A quantidade era desnecessária: cada item referencia uma cópia física específica (um exemplar), e comprar duas cópias gera dois itens.
Pedido	Sem status; total float.	Incluído status; total DECIMAL(10,2).	O status (pendente, pago, enviado, entregue, cancelado) é necessário para o histórico de pedidos e para o cancelamento. O DECIMAL evita erros de arredondamento do float.
Cliente	Sem dados de acesso.	Incluídos senha_hash e perfil.	O sistema tem login e separa clientes de administradores. A senha é guardada cifrada (hash), nunca em texto puro.
Entrega	id_pedido sem restrição.	id_pedido UNIQUE; status com CHECK.	A Iteração 1 definia a relação Pedido e Entrega como 1:1, mas o esquema não a garantia.
Demais tabelas	Campos de status e tipo livres.	Mesma estrutura, com tipos definidos e CHECK nos campos de domínio fechado.	Impede valores inconsistentes (por exemplo, status de pedido escrito de formas diferentes).
Regras de negócio	Previstas "na aplicação ou via trigger", sem definição.	Transações SQL executadas pela aplicação (seção 5).	Define como a regra "exemplar vendido fica indisponível" é garantida. Optou-se por transações simples, sem triggers nem funções, para manter o banco fácil de entender e de demonstrar.

Fonte: elaborado pelos autores (2026).

3 DIAGRAMA ENTIDADE-RELACIONAMENTO E MODELO RELACIONAL

O modelo foi construído na ferramenta dbdiagram.io. O dicionário de dados e as cardinalidades dos relacionamentos aparecem a seguir e, ao final da seção, a Figura 1 apresenta o diagrama lógico da Iteração 2; a versão interativa pode ser acessada pelo link indicado na fonte da figura.

3.1 Dicionário de dados

Quadro 2 – Tabelas do modelo, chaves e papel relacional

Tabela	Chave primária	Chaves estrangeiras	Descrição
Genero	id_genero	Nenhuma	Categorização temática das obras.
Editora	id_editora	Nenhuma	Editoras das obras.
Autor	id_autor	Nenhuma	Dados bibliográficos dos autores.
Obra	id_obra	id_genero, id_editora	Catálogo abstrato da criação intelectual.
Obra_Autor	id_obra, id_autor	id_obra, id_autor	Associativa que resolve o relacionamento N:N entre Obra e Autor.
Exemplar	id_exemplar	id_obra	Unidade física do acervo, com preço, estado e disponibilidade.
Cliente	id_cliente	Nenhuma	Usuários do sistema (clientes e administradores), com endereço.
Telefone	id_telefone	id_cliente	Telefones do cliente (1:N).
Pedido	id_pedido	id_cliente	Compra realizada por um cliente, com status e total.
Item_Pedido	id_item_pedido	id_pedido, id_exemplar	Exemplar vendido em um pedido, com o preço cobrado.
Entrega	id_entrega	id_pedido (único)	Envio do pedido: status, endereço e datas.
Doacao_Troca	id_doacao_troca	id_cliente	Solicitação de doação ou troca de livros.
Item_Doacao	id_item_doacao	id_doacao_troca, id_exemplar (opcional e único)	Livro oferecido, com avaliação e vínculo ao exemplar criado quando aceito.
Mensagem_Contato	id_mensagem	id_cliente (opcional)	Mensagens de clientes ou de visitantes.

Fonte: elaborado pelos autores (2026).

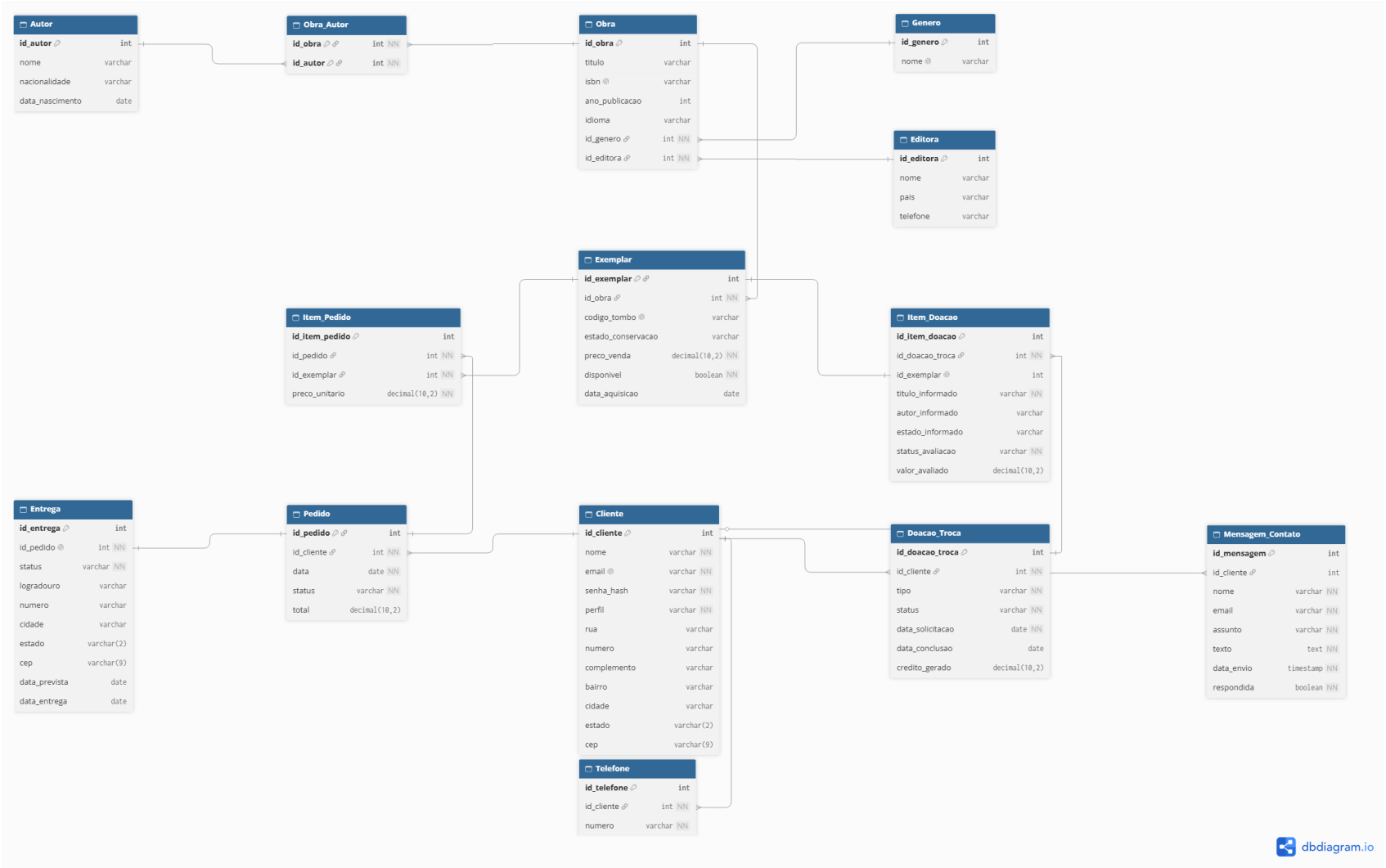
3.2 Cardinalidades dos relacionamentos

Quadro 3 – Cardinalidades dos relacionamentos

Relacionamento	Lado A	Lado B	Leitura
Obra – Exemplar	Obra (0,N)	Exemplar (1,1)	Uma obra tem zero ou vários exemplares; todo exemplar pertence a uma obra.
Obra – Autor	Obra (1,N)	Autor (0,N)	Uma obra tem ao menos um autor; um autor pode ter várias obras (via Obra_Autor).
Obra – Gênero	Obra (1,1)	Gênero (0,N)	Toda obra tem exatamente um gênero.
Obra – Editora	Obra (1,1)	Editora (0,N)	Toda obra tem exatamente uma editora.
Cliente – Telefone	Cliente (0,N)	Telefone (1,1)	Um cliente tem zero ou vários telefones.
Cliente – Pedido	Cliente (0,N)	Pedido (1,1)	Um cliente faz zero ou vários pedidos; todo pedido pertence a um cliente.
Pedido – Item_Pedido	Pedido (1,N)	Item_Pedido (1,1)	Um pedido contém um ou mais itens.
Item_Pedido – Exemplar	Item_Pedido (1,1)	Exemplar (0,N)	Cada item referencia um exemplar. Ao longo do tempo um exemplar pode constar em mais de um item (por exemplo, após o cancelamento de um pedido), mas só em um pedido ativo por vez.
Pedido – Entrega	Pedido (1,1)	Entrega (1,1)	Cada pedido gera exatamente uma entrega.
Cliente – Doacao_Troca	Cliente (0,N)	Doacao_Troca (1,1)	Um cliente faz zero ou várias solicitações.
Doacao_Troca – Item_Doacao	Doacao_Troca (1,N)	Item_Doacao (1,1)	Uma solicitação reúne ao menos um livro, avaliado individualmente.
Item_Doacao – Exemplar	Item_Doacao (0,1)	Exemplar (0,1)	Um item aceito gera no máximo um exemplar; um exemplar vem de no máximo um item doado.
Cliente – Mensagem_Contato	Cliente (0,N)	Mensagem (0,1)	Uma mensagem pode ser de um cliente ou de um visitante sem cadastro.

Fonte: elaborado pelos autores (2026).

Figura 1 – Diagrama lógico integrado do banco de dados do Sebo Literário (Iteração 2)



Fonte: elaborado pelos autores (2026). Disponível em: https://dbdiagram.io/d/trabalho-sebo-iteracao2-6ae63fbf8bf38111a8f7499d.

4 ESTRUTURA DO BANCO DE DADOS (DDL) E POVOAMENTO (DML)

Os scripts seguem a sintaxe do PostgreSQL 15+ e foram testados no PostgreSQL 16. A ordem de execução é DDL_esquema.sql e, em seguida, DML_povoamento.sql.

4.1 Esquema de definição de dados (DDL)

```
-- ============================================================================
-- PROJETO SEBO LITERÁRIO - DDL (ITERAÇÃO 2)
-- SGBD: PostgreSQL 15+
-- ============================================================================

DROP TABLE IF EXISTS Mensagem_Contato CASCADE;
DROP TABLE IF EXISTS Item_Doacao CASCADE;
DROP TABLE IF EXISTS Doacao_Troca CASCADE;
DROP TABLE IF EXISTS Entrega CASCADE;
DROP TABLE IF EXISTS Item_Pedido CASCADE;
DROP TABLE IF EXISTS Pedido CASCADE;
DROP TABLE IF EXISTS Telefone CASCADE;
DROP TABLE IF EXISTS Cliente CASCADE;
DROP TABLE IF EXISTS Exemplar CASCADE;
DROP TABLE IF EXISTS Obra_Autor CASCADE;
DROP TABLE IF EXISTS Obra CASCADE;
DROP TABLE IF EXISTS Autor CASCADE;
DROP TABLE IF EXISTS Genero CASCADE;
DROP TABLE IF EXISTS Editora CASCADE;

-- ----------------------------------------------------------------------------
-- 1. CATÁLOGO
-- ----------------------------------------------------------------------------
CREATE TABLE Autor (
    id_autor        SERIAL PRIMARY KEY,
    nome            VARCHAR(150) NOT NULL,
    nacionalidade   VARCHAR(50),
    data_nascimento DATE
);

CREATE TABLE Genero (
    id_genero SERIAL PRIMARY KEY,
    nome      VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE Editora (
    id_editora SERIAL PRIMARY KEY,
    nome       VARCHAR(100) NOT NULL,
    pais       VARCHAR(50),
    telefone   VARCHAR(20)
);

CREATE TABLE Obra (
    id_obra        SERIAL PRIMARY KEY,
    titulo         VARCHAR(200) NOT NULL,
    isbn           VARCHAR(20) UNIQUE,
    ano_publicacao INT CHECK (ano_publicacao > 1400),
    idioma         VARCHAR(30),
    id_genero      INT NOT NULL REFERENCES Genero(id_genero) ON DELETE RESTRICT,
    id_editora     INT NOT NULL REFERENCES Editora(id_editora) ON DELETE RESTRICT
);

CREATE TABLE Obra_Autor (
    id_obra  INT NOT NULL REFERENCES Obra(id_obra) ON DELETE CASCADE,
    id_autor INT NOT NULL REFERENCES Autor(id_autor) ON DELETE RESTRICT,
    PRIMARY KEY (id_obra, id_autor)
);

CREATE TABLE Exemplar (
    id_exemplar        SERIAL PRIMARY KEY,
    id_obra            INT NOT NULL REFERENCES Obra(id_obra) ON DELETE RESTRICT,
    codigo_tombo       VARCHAR(20) UNIQUE,
```

```
    estado_conservacao VARCHAR(20) CHECK (estado_conservacao IN ('excelente', 'bom',
        'regular', 'danificado')),
    preco_venda        DECIMAL(10,2) NOT NULL CHECK (preco_venda >= 0),
    disponivel         BOOLEAN NOT NULL DEFAULT TRUE,
    data_aquisicao     DATE
);

-- ------------------------------------------------------------------------
-- 2. CLIENTES, PEDIDOS E ENTREGA
-- ------------------------------------------------------------------------
CREATE TABLE Cliente (
    id_cliente  SERIAL PRIMARY KEY,
    nome        VARCHAR(150) NOT NULL,
    email       VARCHAR(120) NOT NULL UNIQUE,
    senha_hash  VARCHAR(255) NOT NULL,
    perfil      VARCHAR(10) NOT NULL DEFAULT 'cliente' CHECK (perfil IN ('cliente',
        'admin')),
    rua         VARCHAR(150),
    numero      VARCHAR(20),
    complemento VARCHAR(100),
    bairro      VARCHAR(80),
    cidade      VARCHAR(100),
    estado      CHAR(2),
    cep         VARCHAR(9)
);

CREATE TABLE Telefone (
    id_telefone SERIAL PRIMARY KEY,
    id_cliente  INT NOT NULL REFERENCES Cliente(id_cliente) ON DELETE CASCADE,
    numero      VARCHAR(20) NOT NULL
);

CREATE TABLE Pedido (
    id_pedido  SERIAL PRIMARY KEY,
    id_cliente INT NOT NULL REFERENCES Cliente(id_cliente) ON DELETE RESTRICT,
    data       DATE NOT NULL DEFAULT CURRENT_DATE,
    status     VARCHAR(15) NOT NULL DEFAULT 'pendente'
               CHECK (status IN ('pendente', 'pago', 'enviado', 'entregue', 'cancelado')),
    total      DECIMAL(10,2) CHECK (total >= 0)   -- atributo derivado (soma dos itens)
);

-- Um mesmo exemplar pode aparecer em mais de um item ao longo do tempo
-- (ex.: pedido cancelado e exemplar vendido depois).
CREATE TABLE Item_Pedido (
    id_item_pedido SERIAL PRIMARY KEY,
    id_pedido      INT NOT NULL REFERENCES Pedido(id_pedido) ON DELETE CASCADE,
    id_exemplar    INT NOT NULL REFERENCES Exemplar(id_exemplar) ON DELETE RESTRICT,
    preco_unitario DECIMAL(10,2) NOT NULL CHECK (preco_unitario >= 0)
);

CREATE TABLE Entrega (
    id_entrega    SERIAL PRIMARY KEY,
    id_pedido     INT NOT NULL UNIQUE REFERENCES Pedido(id_pedido) ON DELETE CASCADE,
    status        VARCHAR(15) NOT NULL DEFAULT 'preparando'
                  CHECK (status IN ('preparando', 'em_transito', 'entregue')),
    logradouro    VARCHAR(150),
    numero        VARCHAR(20),
    cidade        VARCHAR(100),
    estado        CHAR(2),
    cep           VARCHAR(9),
    data_prevista DATE,
    data_entrega  DATE
);

-- ------------------------------------------------------------------------
-- 3. DOAÇÃO/TROCA E CONTATO
-- ------------------------------------------------------------------------
CREATE TABLE Doacao_Troca (
    id_doacao_troca  SERIAL PRIMARY KEY,
    id_cliente       INT NOT NULL REFERENCES Cliente(id_cliente) ON DELETE RESTRICT,
    tipo             VARCHAR(10) NOT NULL CHECK (tipo IN ('doacao', 'troca')),
    status           VARCHAR(15) NOT NULL DEFAULT 'pendente'
```

```
                    CHECK (status IN ('pendente', 'em_avaliacao', 'concluida',
                        'cancelada')),
    data_solicitacao DATE NOT NULL DEFAULT CURRENT_DATE,
    data_conclusao   DATE,
    credito_gerado   DECIMAL(10,2) DEFAULT 0 CHECK (credito_gerado >= 0)  -- derivado; sempre
        0 em doações
);

CREATE TABLE Item_Doacao (
    id_item_doacao   SERIAL PRIMARY KEY,
    id_doacao_troca  INT NOT NULL REFERENCES Doacao_Troca(id_doacao_troca) ON DELETE CASCADE,
    id_exemplar      INT UNIQUE REFERENCES Exemplar(id_exemplar) ON DELETE SET NULL,
    titulo_informado VARCHAR(200) NOT NULL,
    autor_informado  VARCHAR(150),
    estado_informado VARCHAR(20) CHECK (estado_informado IN ('excelente', 'bom', 'regular',
        'danificado')),
    status_avaliacao VARCHAR(10) NOT NULL DEFAULT 'pendente'
                     CHECK (status_avaliacao IN ('pendente', 'aceito', 'recusado')),
    valor_avaliado   DECIMAL(10,2) CHECK (valor_avaliado >= 0)
);

CREATE TABLE Mensagem_Contato (
    id_mensagem SERIAL PRIMARY KEY,
    id_cliente  INT REFERENCES Cliente(id_cliente) ON DELETE SET NULL,
    nome        VARCHAR(150) NOT NULL,
    email       VARCHAR(120) NOT NULL,
    assunto     VARCHAR(100) NOT NULL,
    texto       TEXT NOT NULL,
    data_envio  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    respondida  BOOLEAN NOT NULL DEFAULT FALSE
);
```

4.2 Povoamento e massa de testes (DML)

```
-- ============================================================================
-- POVOAMENTO DA BASE DE DADOS (executar após DDL_esquema.sql)
-- ============================================================================

INSERT INTO Genero (nome) VALUES
('Literatura Brasileira'), ('Distopia'), ('Filosofia'), ('Fantasia');

INSERT INTO Editora (nome, pais, telefone) VALUES
('Companhia das Letras', 'Brasil', '(11) 3707-3500'),
('Editora Record', 'Brasil', '(21) 2585-2000'),
('Nova Fronteira', 'Brasil', '(21) 3882-8200'),
('Martin Claret', 'Brasil', '(11) 3672-8144');

INSERT INTO Autor (nome, nacionalidade, data_nascimento) VALUES
('Machado de Assis', 'Brasileira', '1839-06-21'),
('George Orwell', 'Britânica', '1903-06-25'),
('Clarice Lispector', 'Brasileira', '1920-12-10'),
('J.R.R. Tolkien', 'Britânica', '1892-01-03');

INSERT INTO Obra (titulo, ano_publicacao, idioma, id_genero, id_editora) VALUES
('Memórias Póstumas de Brás Cubas', 1881, 'Português', 1, 1),
('1984', 1949, 'Português', 2, 1),
('A Hora da Estrela', 1977, 'Português', 1, 2),
('O Hobbit', 1937, 'Português', 4, 3);

INSERT INTO Obra_Autor (id_obra, id_autor) VALUES (1, 1), (2, 2), (3, 3), (4, 4);

INSERT INTO Exemplar (id_obra, codigo_tombo, estado_conservacao, preco_venda, disponivel,
    data_aquisicao) VALUES
(1, 'TB-0001', 'bom',       28.50, FALSE, '2026-08-10'),  -- vendido no pedido 1
(1, 'TB-0002', 'excelente', 55.00, TRUE,  '2026-09-02'),
(2, 'TB-0003', 'regular',   19.90, FALSE, '2026-08-15'),  -- vendido no pedido 2
(2, 'TB-0004', 'excelente', 34.00, TRUE,  '2026-09-20'),
(3, 'TB-0005', 'bom',       22.00, TRUE,  '2026-09-25'),
(4, 'TB-0006', 'excelente', 48.00, TRUE,  '2026-10-01');  -- pedido 3 cancelado: segue
    disponível
```

```
-- Os hashes abaixo são apenas ilustrativos (a aplicação gera o hash com bcrypt)
INSERT INTO Cliente (nome, email, senha_hash, perfil, rua, numero, complemento, bairro,
    cidade, estado, cep) VALUES
('Carlos Eduardo Menezes', 'carlos.menezes@email.com',
    '$2b$12$exemploHashCarlos000000000000000000000000000000000000', 'cliente', 'Avenida Senador
    Salgado Filho', '3000', 'Apto 204', 'Candelária', 'Natal', 'RN', '59064-741'),
('Mariana Ferreira Costa', 'mariana.costa@email.com',
    '$2b$12$exemploHashMariana000000000000000000000000000000000000', 'cliente', 'Rua Jaguarari',
    '1450', NULL, 'Lagoa Nova', 'Natal', 'RN', '59054-500'),
('Rodrigo Antunes Lins', 'rodrigo.lins@email.com',
    '$2b$12$exemploHashRodrigo000000000000000000000000000000000000', 'cliente', 'Avenida
    Engenheiro Roberto Freire', '2920', 'Bloco B', 'Capim Macio', 'Natal', 'RN',
    '59082-000'),
('Administrador do Sebo', 'admin@sebo.com',
    '$2b$12$exemploHashAdmin000000000000000000000000000000000000', 'admin', NULL, NULL, NULL,
    NULL, 'Natal', 'RN', NULL);

INSERT INTO Telefone (id_cliente, numero) VALUES
(1, '(84) 99876-1234'), (1, '(84) 3211-0099'), (2, '(84) 98765-4321'), (3, '(84)
    99123-4567');

-- Pedido 1 (entregue): Carlos comprou o exemplar 1
INSERT INTO Pedido (id_cliente, data, status, total) VALUES (1, CURRENT_DATE - 6, 'entregue',
    28.50);
INSERT INTO Item_Pedido (id_pedido, id_exemplar, preco_unitario) VALUES (1, 1, 28.50);
INSERT INTO Entrega (id_pedido, status, logradouro, numero, cidade, estado, cep,
    data_prevista, data_entrega) VALUES
(1, 'entregue', 'Avenida Senador Salgado Filho', '3000', 'Natal', 'RN', '59064-741',
    CURRENT_DATE - 2, CURRENT_DATE - 3);

-- Pedido 2 (pago): Mariana comprou o exemplar 3
INSERT INTO Pedido (id_cliente, data, status, total) VALUES (2, CURRENT_DATE - 1, 'pago',
    19.90);
INSERT INTO Item_Pedido (id_pedido, id_exemplar, preco_unitario) VALUES (2, 3, 19.90);
INSERT INTO Entrega (id_pedido, status, logradouro, numero, cidade, estado, cep,
    data_prevista) VALUES
(2, 'preparando', 'Rua Jaguarari', '1450', 'Natal', 'RN', '59054-500', CURRENT_DATE + 4);

-- Pedido 3 (cancelado): Rodrigo desistiu do exemplar 6, que voltou a ficar disponível
INSERT INTO Pedido (id_cliente, data, status, total) VALUES (3, CURRENT_DATE - 4,
    'cancelado', 48.00);
INSERT INTO Item_Pedido (id_pedido, id_exemplar, preco_unitario) VALUES (3, 6, 48.00);
INSERT INTO Entrega (id_pedido, status, logradouro, numero, cidade, estado, cep) VALUES
(3, 'preparando', 'Avenida Engenheiro Roberto Freire', '2920', 'Natal', 'RN', '59082-000');

-- Doações e trocas
INSERT INTO Doacao_Troca (id_cliente, tipo, status, data_solicitacao, data_conclusao,
    credito_gerado) VALUES
(1, 'troca',   'concluida',    CURRENT_DATE - 10, CURRENT_DATE - 7, 50.00),
(3, 'troca',   'pendente',     CURRENT_DATE - 2,  NULL,             0),
(2, 'doacao',  'em_avaliacao', CURRENT_DATE - 1,  NULL,             0);

INSERT INTO Item_Doacao (id_doacao_troca, id_exemplar, titulo_informado, autor_informado,
    estado_informado, status_avaliacao, valor_avaliado) VALUES
(1, 2,    'Memórias Póstumas de Brás Cubas - Capa Dura', 'Machado de Assis', 'excelente',
    'aceito',   50.00),
(1, NULL, 'Dom Casmurro - exemplar com páginas soltas',  'Machado de Assis', 'danificado',
    'recusado', NULL),
(2, NULL, '1984',                                        'George Orwell',    'bom',
    'pendente', NULL),
(3, NULL, 'Cálculo com Geometria Analítica',             'Louis Leithold',   'regular',
    'pendente', NULL);

-- Contato (um cliente e um visitante sem cadastro)
INSERT INTO Mensagem_Contato (id_cliente, nome, email, assunto, texto, respondida) VALUES
(2,    'Mariana Ferreira Costa', 'mariana.costa@email.com', 'Dúvida sobre envio', 'A entrega
    é feita em horário comercial pelos Correios?', TRUE),
(NULL, 'Lucas Silveira', 'lucas.visitante@gmail.com', 'Sugestão de livros', 'Vocês aceitam
    coleções completas de quadrinhos na modalidade troca?', FALSE);
```

5 REGRAS DE NEGÓCIO IMPLEMENTADAS PELA APLICAÇÃO

Neste projeto, as regras de negócio ficam na aplicação, que executa cada operação como uma transação SQL (BEGIN ... COMMIT), sem triggers nem funções no banco. Se qualquer passo falhar, a aplicação desfaz tudo com ROLLBACK. Os exemplos desta seção estão no arquivo Transacoes_exemplo.sql do repositório, que ao ser executado imprime, para cada transação, a situação, os passos, o estado antes e depois e o significado no contexto do sebo. Nos exemplos o desfecho é ROLLBACK, para preservar os dados; na aplicação seria COMMIT.

5.1 Compra de um livro

Situação: Carlos (cliente 1) compra o exemplar 4, o livro "1984" em estado excelente, com preço de tabela de R$ 34,00. A transação reserva o exemplar, cria o pedido, cria o item copiando o preço atual do exemplar, calcula o total e cria a entrega com o endereço do cliente.

No sebo: o livro sai da vitrine para ninguém mais comprá-lo, e o preço pago fica gravado no pedido mesmo que o sebo remarque o livro depois.

```
BEGIN;
-- 1. Reserva o exemplar (só se ainda estiver disponível)
UPDATE Exemplar SET disponivel = FALSE WHERE id_exemplar = 4 AND disponivel = TRUE;
-- 2. Cria o pedido
INSERT INTO Pedido (id_cliente, status) VALUES (1, 'pendente');
-- 3. Cria o item, copiando o preço atual do exemplar (preço congelado)
INSERT INTO Item_Pedido (id_pedido, id_exemplar, preco_unitario)
SELECT currval('pedido_id_pedido_seq'), id_exemplar, preco_venda FROM Exemplar WHERE
    id_exemplar = 4;
-- 4. Calcula o total do pedido
UPDATE Pedido SET total = (SELECT SUM(preco_unitario) FROM Item_Pedido
                           WHERE id_pedido = currval('pedido_id_pedido_seq'))
WHERE id_pedido = currval('pedido_id_pedido_seq');
-- 5. Cria a entrega com o endereço do cliente
INSERT INTO Entrega (id_pedido, logradouro, numero, cidade, estado, cep)
SELECT currval('pedido_id_pedido_seq'), rua, numero, cidade, estado, cep FROM Cliente WHERE
    id_cliente = 1;
COMMIT;
```

5.2 Tentativa de comprar um livro que já foi vendido

Situação: alguém tenta comprar o exemplar 1, que Carlos já comprou no pedido 1. O UPDATE condicional não reserva nenhuma linha (retorna 0 linhas), a aplicação percebe isso, dá ROLLBACK e avisa o cliente.

No sebo: como cada exemplar é uma cópia física individual, a mesma cópia nunca é vendida duas vezes, nem se duas pessoas clicarem em "comprar" ao mesmo tempo, porque a linha fica bloqueada durante a atualização.

```
BEGIN;
UPDATE Exemplar SET disponivel = FALSE WHERE id_exemplar = 1 AND disponivel = TRUE RETURNING
    id_exemplar;
-- Resultado: 0 linhas. Compra recusada.
ROLLBACK;
```

5.3 Cancelamento de pedido

Situação: Mariana (cliente 2) desiste do pedido 2, que estava pago. A transação muda o status do pedido para cancelado e devolve os exemplares dele à vitrine.

No sebo: o livro volta a poder ser vendido a outro cliente, e o pedido continua no histórico, apenas marcado como cancelado. Como o Item_Pedido não é único por exemplar, o livro pode ser vendido de novo.

```
BEGIN;
UPDATE Pedido SET status = 'cancelado' WHERE id_pedido = 2;
UPDATE Exemplar SET disponivel = TRUE
WHERE id_exemplar IN (SELECT id_exemplar FROM Item_Pedido WHERE id_pedido = 2);
COMMIT;
```

5.4 Aceite de um livro oferecido em troca

Situação: Rodrigo (cliente 3) ofereceu o livro "1984" em troca (solicitação 2, item 3), e o administrador o aceitou, atribuindo R$ 15,00 de crédito. A transação cadastra o livro como novo exemplar, marca o item como aceito e vinculado a esse exemplar, e atualiza a solicitação, cujo credito_gerado passa a ser a soma dos itens aceitos.

No sebo: o livro passa a fazer parte do acervo e o cliente fica com R$ 15,00 de crédito registrado. Em uma doação o livro também entraria no acervo, mas o crédito continuaria 0. O abatimento do crédito em compras futuras fica fora do escopo desta iteração.

```
BEGIN;
-- 1. Cadastra o livro aceito como novo exemplar do acervo
INSERT INTO Exemplar (id_obra, codigo_tombo, estado_conservacao, preco_venda, data_aquisicao)
VALUES (2, 'TB-0007', 'bom', 25.00, CURRENT_DATE);
-- 2. Marca o item como aceito e o liga ao novo exemplar
UPDATE Item_Doacao
   SET status_avaliacao = 'aceito', valor_avaliado = 15.00,
       id_exemplar = currval('exemplar_id_exemplar_seq')
 WHERE id_item_doacao = 3;
-- 3. credito_gerado = soma dos itens aceitos (só em troca)
UPDATE Doacao_Troca
   SET credito_gerado = (SELECT COALESCE(SUM(valor_avaliado), 0) FROM Item_Doacao
                         WHERE id_doacao_troca = 2 AND status_avaliacao = 'aceito'),
       status = 'concluida', data_conclusao = CURRENT_DATE
 WHERE id_doacao_troca = 2 AND tipo = 'troca';
COMMIT;
```

6 EXEMPLOS DE UTILIZAÇÃO E CONSULTAS PRÁTICAS

A seguir estão cinco consultas analíticas, que ficam em Consultas_analiticas.sql. Ao executar o arquivo, cada consulta imprime antes do resultado um texto com as tabelas usadas, o que ela mostra, como ler o resultado e para que serve no sebo (os textos são omitidos nos códigos abaixo). Os resultados foram obtidos executando os scripts da seção 4 em um banco novo.

6.1 Consulta 1: catálogo com exemplares disponíveis e faixas de preço

Lista o catálogo público, com autores, gênero e editora, a quantidade de exemplares disponíveis e as faixas de preço. Cada linha é uma obra. No sebo, é a vitrine do site.

```
SELECT
    o.id_obra,
    o.titulo,
    STRING_AGG(DISTINCT a.nome, ', ') AS autores,
    g.nome AS genero,
    ed.nome AS editora,
    COUNT(DISTINCT e.id_exemplar) AS exemplares_disponiveis,
    MIN(e.preco_venda) AS menor_preco,
    MAX(e.preco_venda) AS maior_preco
FROM Obra o
JOIN Genero g ON o.id_genero = g.id_genero
JOIN Editora ed ON o.id_editora = ed.id_editora
JOIN Obra_Autor oa ON o.id_obra = oa.id_obra
JOIN Autor a ON oa.id_autor = a.id_autor
JOIN Exemplar e ON o.id_obra = e.id_obra
WHERE e.disponivel = TRUE
GROUP BY o.id_obra, o.titulo, g.nome, ed.nome
ORDER BY o.titulo;
```

Quadro 4 – Resultado da Consulta 1 (coluna id_obra omitida)

titulo	autores	genero	editora	exemplares_ disponiveis	menor_ preco	maior_ preco
1984	George Orwell	Distopia	Companhia das Letras	1	34.00	34.00
A Hora da Estrela	Clarice Lispector	Literatura Brasileira	Editora Record	1	22.00	22.00
Memórias Póstumas de Brás Cubas	Machado de Assis	Literatura Brasileira	Companhia das Letras	1	55.00	55.00
O Hobbit	J.R.R. Tolkien	Fantasia	Nova Fronteira	1	48.00	48.00

Fonte: elaborado pelos autores (2026).

6.2 Consulta 2: histórico de pedidos de um cliente

Exibe os pedidos de um cliente (id_cliente = 1, Carlos Eduardo Menezes), com a obra comprada, o preço pago, o total e a situação da entrega. No sebo, é a tela "Meus pedidos".

```
SELECT
    p.id_pedido, p.data, p.status,
    o.titulo AS obra, e.estado_conservacao,
    ip.preco_unitario, p.total,
    en.status AS status_entrega, en.data_prevista, en.data_entrega
FROM Pedido p
JOIN Item_Pedido ip ON p.id_pedido = ip.id_pedido
JOIN Exemplar e ON ip.id_exemplar = e.id_exemplar
JOIN Obra o ON e.id_obra = o.id_obra
LEFT JOIN Entrega en ON p.id_pedido = en.id_pedido
WHERE p.id_cliente = 1
ORDER BY p.data DESC;
```

Resultado: uma linha, com o pedido 1 (status entregue), a obra "Memórias Póstumas de Brás Cubas" em estado bom, preço pago e total de 28.50 e entrega com status entregue.

6.3 Consulta 3: faturamento e vendas por gênero

Permite à gestão ver quais gêneros faturam mais nos pedidos pagos, enviados ou entregues. Pedidos pendentes ou cancelados não contam como venda. No sebo, apoia a decisão de quais gêneros buscar.

```
SELECT
    g.nome AS genero,
    COUNT(ip.id_item_pedido) AS livros_vendidos,
    SUM(ip.preco_unitario) AS faturamento,
    ROUND(AVG(ip.preco_unitario), 2) AS ticket_medio
FROM Item_Pedido ip
JOIN Pedido p ON ip.id_pedido = p.id_pedido
JOIN Exemplar e ON ip.id_exemplar = e.id_exemplar
JOIN Obra o ON e.id_obra = o.id_obra
JOIN Genero g ON o.id_genero = g.id_genero
WHERE p.status IN ('pago', 'enviado', 'entregue')
GROUP BY g.nome
ORDER BY faturamento DESC;
```

Quadro 5 – Resultado da Consulta 3

genero	livros_vendidos	faturamento	ticket_medio
Literatura Brasileira	1	28.50	28.50
Distopia	1	19.90	19.90

Fonte: elaborado pelos autores (2026).

6.4 Consulta 4: doações e trocas e crédito gerado

Acompanha as solicitações, quantos itens foram enviados e aceitos e o crédito gerado. Só trocas geram crédito; em doações ele é sempre 0. No sebo, é a fila de trabalho do administrador.

```
SELECT
    dt.id_doacao_troca,
    c.nome AS cliente,
    dt.tipo,
    dt.status,
    COUNT(it.id_item_doacao) AS itens_enviados,
    COUNT(*) FILTER (WHERE it.status_avaliacao = 'aceito') AS itens_aceitos,
    dt.credito_gerado
FROM Doacao_Troca dt
JOIN Cliente c ON dt.id_cliente = c.id_cliente
LEFT JOIN Item_Doacao it ON dt.id_doacao_troca = it.id_doacao_troca
GROUP BY dt.id_doacao_troca, c.nome, dt.tipo, dt.status, dt.credito_gerado
ORDER BY dt.data_solicitacao;
```

Quadro 6 – Resultado da Consulta 4

id_doacao_ troca	cliente	tipo	status	itens_ enviados	itens_ aceitos	credito_ gerado
1	Carlos Eduardo Menezes	troca	concluida	2	1	50.00
2	Rodrigo Antunes Lins	troca	pendente	1	0	0.00
3	Mariana Ferreira Costa	doacao	em_avaliacao	1	0	0.00

Fonte: elaborado pelos autores (2026).

6.5 Consulta 5: estoque disponível por estado de conservação

Mostra a proporção de exemplares disponíveis por estado de conservação e a faixa de preços. No sebo, auxilia a precificação e a escolha do que aceitar em trocas.

```
SELECT
    e.estado_conservacao,
    COUNT(*) AS quantidade,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM Exemplar WHERE disponivel = TRUE), 2) AS
        percentual,
    MIN(e.preco_venda) AS menor_preco,
    ROUND(AVG(e.preco_venda), 2) AS preco_medio,
    MAX(e.preco_venda) AS maior_preco
FROM Exemplar e
WHERE e.disponivel = TRUE
GROUP BY e.estado_conservacao
ORDER BY quantidade DESC;
```

Quadro 7 – Resultado da Consulta 5

estado_ conservacao	quantidade	percentual	menor_ preco	preco_ medio	maior_ preco
excelente	3	75.00	34.00	45.67	55.00
bom	1	25.00	22.00	22.00	22.00

Fonte: elaborado pelos autores (2026).